

10th International Conference of Information and Communication Technology (ICICT-2020)

Design and implementation of neural network computing framework on Zynq SoC embedded platform

Xingying Li^{a,b}, Zhenyu Yin^{a,b,*}, Fulong Xu^{a,b}, Feiqing Zhang^{a,b}, Guangyuan Xu^{a,b}

^aShenyang Institute of Computing Technology Chinese Academy of Science, Shenyang, China

^bUniversity of Chinese Academy of Sciences, Beijing, China

Abstract

Limited resources and low computing power of embedded platform make it difficult to apply neural network technology. To overcome this problem, a new neural network computing framework “Zynq-Darknet” was proposed. The framework is based on Darknet, which constructs depthwise separable convolution and a lightweight classification model MobileNetV2 and was deployed to Xilinx Zynq-7000 System-on-Chip (SoC) with Linux operating system (OS). In order to verify the performance of the framework and model, experiments were conducted on imagenet-1k dataset using different network structures. The results show that the MobileNetV2 network model based on Zynq-Darknet can effectively perform image classification, and ensure a certain real-time and accuracy while reducing the computational complexity and storage overhead, assuming promising application prospects.

© 2021 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the 10th International Conference of Information and Communication Technology.

Keywords: Neural network; embedded platform; Zynq SoC; darknet; depthwise separable convolution; MobileNetV2;

1. Introduction

With the rapid popularization of the Internet of Things and smart edge devices, there is an increasing demand for implementing machine learning algorithms on embedded devices^[1]. Neural network technology has been continuously extended to embedded platforms, such as fault prediction, object detection and recognition system in industrial environment. However, the existing neural network computing framework is often accompanied with a lot of memory overhead and complex computation while improving the accuracy. How to deploy neural network

* Corresponding E-mail address: congmy@163.com.

computing framework in embedded platform with limited resources and computing capacity has become a difficult problem that seriously hinders the application and development of neural network technology in embedded platform. Currently, some deep learning applications are deployed on GPU-based embedded systems, such as smartphones and tablets, but most of these applications still rely on uploading data over the Internet to cloud servers for computing. This cloud-based client-server model has the problems of high latency, high network overhead and power consumption, as well as privacy issues, making it difficult to apply to actual business application scenarios^[2]. Therefore, balancing computing power and memory overhead, building an embedded neural network computing framework with excellent performance on an embedded mobile system, and deploying an offline neural network model are the keys to promote the deployment of neural network on an embedded platform.

In order to meet the "three low" requirements of low cost, low power consumption and low latency of neural network computing in embedded platforms, we select Zynq-7020 SoC as the embedded development platform in this paper, build Zynq-Linux OS, introduce depthwise separable convolution and construct a lightweight classification model MobileNetV2 based on lightweight open-source neural network framework Darknet, so as to implement the Zynq-Darknet. Finally, the ImageNet data set is used to train the weights on the PC side, and the framework is experimentally verified on the Zynq-7000 platform. Our experimental results show that the MobileNetV2 network model based on our Zynq-Darknet framework only needs 12.731s for single image recognition, which can reach 90.3% of top-5 accuracy with the computational complexity being only 0.858BLOPFs and the number of parameters being only 14.2MB, achieving a good balance between computing power, real-time and accuracy.

2. Related work

At present, GPU and TPU show superior performance in neural network calculations and become the first-choice hardware for processing problems with neural networks. However, due to their extremely high power consumption, they are difficult to use on embedded platforms. Zynq-7000 SoC is a programmable chip system produced by Xilinx, which combines processor system (PS) with programmable logic (PL) in a single chip. The processor system includes a dual-core ARM processor, memory interface and other devices, while the PL part is the traditional FPGA, which can realize the hardware acceleration of the algorithm. Zynq SOC has high flexibility, scalability and low power consumption. It can replace GPU or TPU as the platform of embedded neural network deployment^[3].

The neural network computing framework constructs the underlying algorithm of the neural network in the form of components. Users can quickly build a neural network model based on the network structure, which greatly reduces the threshold for studying neural network technology and simplifies the development process of neural network^[4]. In recent years, academia has put forward many related researches on lightweight neural network computing frameworks for embedded platforms. Li et al^{[5][6]} proposed a lightweight neural network which supports multiple languages and has done a lot of optimizations for the mobile terminal^[7]; He et al^{[8][9]} proposed a framework makes full use of the heterogeneous computing power of mobile chips and has extensive hardware support. The literature [10] launched a framework focuses on supporting mobile end inference prediction, which is characterized by high performance, multiple hardware, and lightweight. J.Redmon^[11], proposed an open source neural network computing framework based on C language and CUDA, which has the characteristics of fast speed and easy installation. Moreover, compared with the above frameworks, the framework has no dependence, strong portability, and is convenient for improvement and expansion^[12]. It is an ideal framework for the development and deployment of embedded platform^[12].

To promote the application of neural network technology in embedded platforms, we should not only make full use of the computing power of the equipment to speed up the calculation of the network, but also design a network structure with less parameters and calculation for the embedded platform. The lightweight neural network model proposed in reference [13][14] adopts the idea of depthwise separable convolution^[15] to reduce the amount of parameters generated in the process of convolution, reduce the size and complexity of the model, and achieve good results in experiments. Besides, the network model proposed in document [16][17] also draws on the idea of depthwise separable convolution, but the network model in [17] adds inverted residual and linear bottleneck structure, which reduces the amount of parameters and computation while ensuring the expressive power of the network, and is widely studied and applied.

In view of the application requirements of neural network technology in embedded platforms, as well as the advantages of Zynq SoC and Darknet, we have optimized and extended it based on the Darknet, and deploys it to Zynq SOC platform with Linux OS to realize an embedded neural network computing framework Zynq-Darknet which is oriented to Zynq SoC. Experiments have proved that using this framework to realize neural network calculations guarantees certain real-time performance and recognition accuracy on the basis of reducing the amount of calculation. It can be used as an effective tool for neural network applications on embedded platforms.

3. Implementation

Zynq-7020 development board was selected as the experimental platform in this paper. First, we built the embedded Linux OS on the development board, then optimized and extended it based on the Darknet open-source framework to replace the traditional convolution with depthwise separable convolution and implement the lightweight classification model MobileNetV2. Finally, we deploy it to the development board to get our Zynq-Darknet.

3.1. Build the Linux OS of Zynq embedded platform

The design and development of embedded system can be divided into two modes: bare machine application development and embedded OS development. The former is oriented to hardware development, and users can operate the physical hardware directly through the address, which is more difficult to develop. The latter is oriented to OS development and manages the software and hardware resources through the OS, making the embedded system design more flexible and convenient, effectively reducing the development difficulty and improving the development efficiency.

In this paper, Zynq embedded Linux OS is constructed by OS-oriented development method, including hardware configuration and system development. Hardware configuration is the condition for Zynq-7020 development board to start Linux; System development is the core of constructing Linux OS on Zynq-7020 development board, which mainly includes the establishment of cross-compilation environment and the compilation of U-Boot, Kernel, Devicetree and Rootfs^[18]. Among them, cross-compilation environment provides conditions for the compilation of OS software to ensure the smooth construction of Linux; U-boot is responsible for simple initialization of the device; Linux kernel is the core of the OS, being responsible for the process management, memory management and other functions of the system; Devicetree describes the hardware device connected to Zynq in the form of a tree, which reduces the heavy dependence of the kernel on the system hardware and makes the design and compilation of the kernel easier. Rootfs provide the OS with a way to store data on storage devices. The construction process of Zynq embedded Linux OS is shown in Fig. 1.

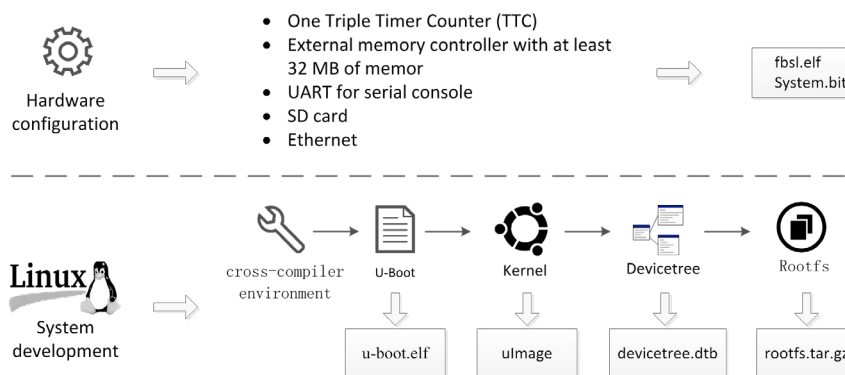


Fig. 1. Construct Linux OS on Zynq embedded

3.2. Convolution optimization

A complete neural network usually consists of multiple convolution layers, pooling layers and full connection layers. Convolution layer is the core layer of neural network, which generates most of the computation in the network. The traditional convolution is used in the Darknet, which will produce a lot of parameters in the calculation process, increasing the storage cost and computational complexity. Therefore, depthwise separable convolution is used to replace the traditional convolution for optimization in the paper.

Depthwise separable convolution decomposes the convolution operation into depthwise convolution and 1×1 pointwise convolution^[16]. Different from the convolution kernel of traditional convolution, depthwise convolution adopts different convolution kernels for each input channel, that is, one convolution kernel is responsible for one input channel and extracts the information of each channel. Pointwise convolution is similar to the traditional convolution, which can combine the feature information of different channels at the same position to generate a new feature map. It can effectively reduce the amount of parameters and complexity of neural network calculation by filtering the feature graph through depthwise convolution and merging the information by pointwise convolution.

The traditional convolution computation process is shown in Fig. 1, while Fig. 3 is the computation process of depthwise separable convolution, which shows how a traditional convolution is factorized into a depthwise convolution 3(a) and a 1×1 pointwise convolution 3(b). For the convolution calculation of: a $D_F * D_F * M$ channels input and $D_K * D_K * M$ filter, to produce a $D_F * D_F * N$ feature map,

traditional convolutions have the computational cost of:

$$D_K * D_K * M * N * D_F * D_F$$

depthwise separable convolution have the cost of:

$$D_K * D_K * M * D_F * D_F + M * N * D_F * D_F$$

From this, it can be obtained that under the same situation, the cost ratio of depthwise separable convolution calculation to traditional convolution calculation is

$$1/N + 1/D_K^2 < 1$$

It can be seen that using depthwise separable convolution in convolution calculation can reduce the calculation amount to $1/D_K^2$ compared with traditional convolution.

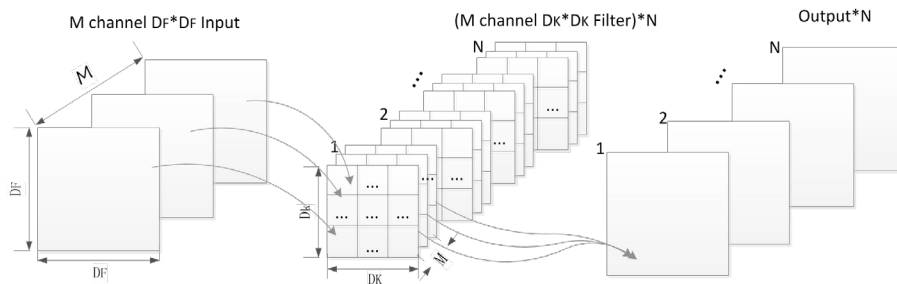


Fig. 2. Traditional convolution

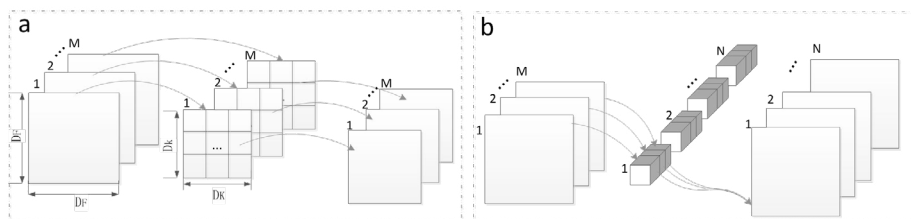


Fig. 3. Depthwise separable convolution (a) depthwise convolution; (b) 1×1 pointwise convolution

3.3. Build the MobileNeV2 network model

At present, neural network based image classification technology has achieved good results in various fields. However, due to the constant pursuit of accuracy, the structure of neural network model is getting deeper and more complex^[19]. However, in the actual embedded application scenario, the huge model is difficult to deploy and apply. First of all, the huge model needs more memory, and then it will cause slow calculation speed, which is difficult to adapt to the actual application of embedded system. Therefore, designing and deploying a small and efficient neural network model for an embedded platform is very important for the implementation of embedded neural network technology. On the basis of Darknet, we add a lightweight classification model MobileNetV2 with depthwise separable convolution, linear bottleneck and inverted residual structure, to apply image classification technology in Zynq embedded platform. The network structure of MobileNetV2 is shown in Table 1.

Table 1. Network structure of MobileNetV2.

Input	Operate	t	c	n	s
224*224*3	conv2d	-	32	1	2
112*112*32	bottleneck	1	16	1	1
112*112*16	bottleneck	6	24	2	2
56*56*24	bottleneck	6	32	3	2
28*28*32	bottleneck	6	64	4	2
14*14*64	bottleneck	6	96	3	1
14*14*96	bottleneck	6	160	3	2
7*7*160	bottleneck	6	320	1	1
7*7*320	conv2d 1*1	-	1280	1	1
7*7*1280	avgpool 7*7	-	-	1	-
1*1*1280	conv2d 1*1	-	k	-	-

Using depthwise separable convolution with residuals, bottleneck is the basic building block of the MobileNetV2^[17]. This block firstly expands the number of channels in the input feature map with 1*1 convolution, and then extracts the features with depthwise convolution. Finally, it reduces to the original dimension with 1*1 convolution, and adds jump connection in a specific block, so that the network can still be trained at a deeper level. In order to guarantee the expression ability of the model and solve the problem of low-dimensional information loss caused when activation function is ReLU, linear bottleneck structure (linear activation function replace ReLU in low dimensional feature map) is also used. The basic implementation of the bottleneck block is shown in Fig.4.

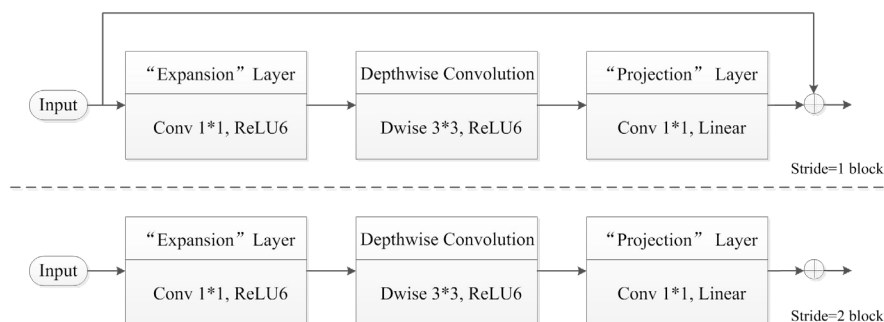


Fig. 4. The basic implementation of bottleneck.

4. Experiment and result analysis

4.1. Experimental environment

Since Zynq SoC has few resources, we use PC to train the model and download the weight to Zynq-7020 embedded platform for testing. The specific experimental environment is shown in Table 2.

Table 2. Experimental environment.

Type	PC	Zynq-7020
OS	Ubuntu18.04.01	Ubuntu18.04.01
CPU	Intel(R) Core(TM) i5-8265U	ARM Cortex-A9
DDR	8G DDR3 1600MHZ	1G DDR3 1066MHZ

4.2. Experiment

In this paper, the MobileNetV2 based on Zynq-Darknet framework was compared with AlexNet and Darknet-19 on ImageNet-1K image classification data set. During the experiment, all the conditions remained the same except the different network structure. We compared the five indicators of top-1, top-5, recognition time of single picture, computational complexity and number of parameters of different networks, the results are shown in Table 3. It can be seen that the time used by MobileNetV2 to recognize a single image is only 12.731s, and higher recognition accuracy has been maintained. In addition, compared with other network structures, the computational complexity and storage overhead are lower.

Table 3. Performance of different networks.

Data set	Network	Top-1	Top-5	Times	Ops	Params
Imagenet-1k	AlexNet	72.9	91.2	25.472s	2.153Bn	249.5MB
	Darknet-19	70.7	89.9	83.936s	7.291Bn	83.4MB
	Mobilenetv2	71.4	90.3	12.731s	0.858Bn	14.2MB

5. Conclusion

In order to apply neural network technology to embedded platform, the Zynq-Darknet, a neural network computing framework with depthwise separable convolution and lightweight classification model MobileNetV2 was built according to the characteristics of less resources and low computing power of embedded platforms, basing on Darknet and deploying on Zynq -7020 development board. The Zynq-Darknet framework is applied to establish different neural network models for image recognition experiments on ImageNet-1k data sets in our experiment. The result shows that the framework can be applied on the Zynq embedded platform. Additionally, the network model MobileNetV2 based on the framework shows great advantage over other network models with higher recognition rate and accuracy and lower computational complexity and system storage overhead, having great advantages.

Acknowledgement

This work is supported by Science and Technology Program of Liaoning Province under Grant NO. 2019JH2/10100019. It is also supported by National Key Research and Development Project under Grant NO. 2017YFE0125300.0

References

- [1] Meng W , Gu Z , Zhang M , et al. Two-Bit Networks for Deep Learning on Resource-Constrained Embedded Devices[J]. *arXiv:1701.00485 [cs.LG]*, 2017.

- [2] Li, Qian , Q. Xiao , and Y. Liang . Enabling high performance deep learning networks on embedded systems. *IECON 2017 - 43rd Annual Conference of the IEEE Industrial Electronics Society*. IEEE, 2017.
- [3] Zhang, Yunke, L. Dan. Realtime implementation of improved TINY YOLO vehicle detection algorithm based on Zynq SoC hardware acceleration[J]. *Journal of Computer Applications*, 2019
- [4] Mohammad Loni, Sima Sinaei, Ali Zoljodi, Masoud Daneshlab, Mikael Sjödin. DeepMaker: A multi-objective optimization framework for deep neural networks in embedded systems [J]. *Microprocessors and Microsystems*, 2020.
- [5] TensorFlow Lite guide [EB/OL]. <https://tensorflow.google.cn/lite/guide>, Accessed on 2020-09-23.
- [6] Li, Shuangfeng. TensorFlow Lite: On-Device Machine Learning Framework [J]. *Journal of Computer Research and Development*, 2020
- [7] Gao, Luchuan. Research and analysis of TensorFlow Lite development platform [J]. *Electronic Test*, 2018.
- [8] He, Liangliang. Practice and Thinking of Xiaomi MACE Open Source Project [J]. *Software and Integrated Circuit*, 2019(04): 36-39.
- [9] Zhao, qun, L. tuo, H. Liangliang. Z. Ming, C. Baoqiu. An Open-Source Neural Network Framework for Mobile Heterogeneous Computing MACE. *Information Technology & Standardization*, 2020 (04): 26-29.
- [10] Paddle Lite [EB/OL]. <https://github.com/PaddlePaddle/Paddle-Lite>, Accessed on 2020-09-23.
- [11] J. Redmon. Darknet: Open Source Neural Networks in C [EB/OL]. <https://pjreddie.com/darknet>, Accessed on 2020-09-23.
- [12] T. B. Preußer, G. Gambardella, N. Fraser and M. Blott. Inference of quantized neural networks on heterogeneous all-programmable devices[C]. *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. Dresden, 2018, pp. 833-838.
- [13] Chollet F . Xception: Deep Learning with Depthwise Separable Convolutions[C]. *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2017.
- [14] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, et al. ShuffleNet: An Extremely Efficient Convolutional Neural Network for Mobile Devices. *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 6848-6856.
- [15] Sifre L., Mallat, Stéphane. Rigid-Motion Scattering for Texture Classification[J]. *Computer ence*, 2014, 3559:501-515.
- [16] Andrew G. Howard, Menglong Zhu, Bo Chen, et al. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv: 1704.04861 [cs. CV]*. Apr 2017.
- [17] Sandler M, Howard A, Zhu M, et al. Mobilenetv2: Inverted residuals and linear bottlenecks[C]. *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018: 4510-4520.
- [18] Jigneshkumar J. Patel, Nagaraj Reddy, Praveena Kumari, Rachana Rajpal, Harshad Pujara, R. Jha, Praveen Kalappurakkal. Embedded Linux platform for data acquisition systems. *Fusion Engineering and Design*, 2018.
- [19] Bi, Sheng, et al. An Embedded Inference Framework for Convolutional Neural Network Applications [J]. *IEEE Access* 7, 2019.